T12 MCP Servers Overview

Technical Intermediate

The extension protocol and essential servers

What is MCP?

MCP (Model Context Protocol) is the open standard that allows Claude Code to connect to external tools via a JSON-RPC 2.0 protocol. Each MCP server exposes additional tools, named according to the `mcp__<server>__<tool>` convention. Claude uses them exactly like its native tools, with the same permissions system.

Basic architecture: Claude Code (client) communicates with the MCP server via stdio or HTTP. Connection timing and reconnection depend on the server type, runtime and configuration; inspect `/mcp` for current status.

Most widely used servers

Server	Primary use
Context7	Official library documentation
Sequential Thinking	Structured multi-step reasoning
Grepai	Semantic search + call graph
Serena	Symbol navigation, refactoring
Playwright	Browser automation, E2E tests
GitHub	Issues, PRs, repositories

MCP configuration scopes

Use `claude mcp add` for the desired scope. Project servers are stored in `.mcp.json`; user and local configurations are stored in `~/.claude.json`.

```
{
  "mcpServers": {
    "context7": {
      "command": "npx",
      "args": ["-y", "@upstash/context7-mcp", "--api-key",
"YOUR_API_KEY"],
      "env": {}
    },
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    }
  }
}
```

Authenticate the GitHub remote server through `/mcp` before use; account and organization policies may restrict access.

Transport: local (stdio) vs remote (HTTP)

stdio (local): the server runs as a child process on the machine. It can still make outbound network requests; review its code and permissions.

HTTP/SSE (remote): the server is accessible via a URL. Used for hosted services like Figma MCP (`https://mcp.figma.com/mcp`) or shared team servers.

```
{
  "mcpServers": {
    "figma": {
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
  }
}
```

MCP Apps (SEP-1865)

Extension available since January 26, 2026, co-authored by Anthropic, OpenAI, and the MCP-UI creators. It allows MCP servers to return interactive interfaces (HTML/JS) in addition to classic text responses.

Not available in Claude Code CLI: the terminal stays text-only, these interfaces do not render there. MCP Apps works in Claude Desktop, VS Code Insiders, and ChatGPT. Value for a CLI user: explore visually in Claude Desktop, then automate the identified actions in Claude Code.

alwaysLoad (v2.1.121)

By default, MCP tools are loaded on demand (via `ToolSearch`). For servers with 1 to 5 critical tools needed every session, use `alwaysLoad: true` to make them always available without a prior search call:

```
{
  "mcpServers": {
    "my-critical-server": {
      "command": "npx",
      "args": ["my-mcp-server"],
      "alwaysLoad": true
    }
  }
}
```

Avoid on servers with many tools (20+): the token cost adds up quickly.

Important limitations

An MCP server sees only the parameters passed at call time, not the conversation history. Each call is independent unless the server itself implements a cache. It also cannot modify Claude's system prompt or bypass the permissions system.

Token cost: tool-definition overhead varies with the schemas and tool-search deferral. Load only what is necessary for the current project.